Wyrd III Tutorial
Projects, VTR, Simulation and Midgard III

For Wyrd III 0.1 (Yggdrasil III suite) *October 2026*

Contents

1 Introduction **3**

1.1 What you need . . . 3

2 Installing and Configuring Wyrd III **3**

2.1 Installing . . . 3

2.2 Describing your machine . . . 4

3 Making Your Own Project **5**

3.1 The Verilog design . . . 5

3.2 Creating wyrd.toml . . . 6

3.3 When the Verilog is wrong: a hard-cell name . . . 7

3.4 Running the project . . . 7

3.5 Reading the results table . . . 8

3.6 What is in the run directory . . . 8

3.7 Comparing two versions of a design . . . 10

4 The Graphical Interface **10**

4.1 Opening the project . . . 10

4.2 Editing wyrd.toml . . . 11

4.3 Running the flow . . . 13

4.4 Reading the report . . . 14

5 Simulating the Design **16**

5.1 The floorplan viewer . . . 16

5.2 From the command line . . . 16

5.3 The Simulate tab . . . 17

6 The Populated Samples **20**

7 Midgard III (Verilog Town) 21

7.1 Starting the game 21

7.2 Placing a sensor 22

7.3 Editing the Verilog of a light 23

7.4 Running the level 26

7.5 The level editor 27

8 Where to Go Next 28

1 Introduction

This tutorial shows how to use Wyrd III, the experiment runner and workbench around the Odin III synthesis front end. You will build a small project of your own from an empty folder, run it through Odin III and VTR, read the results, look at it in the graphical interface, simulate it cycle by cycle, and then use the ready-made content that comes with Wyrd III: the sample project and the Midgard III game.

Doing this tutorial, the reader will learn about:

- Installing Wyrd III and describing your machine to it
- Writing a Verilog design and a `wyrd.toml` project file
- Running the design through Odin III and VPR for two FPGA architectures
- Reading the results table and the run directory, and comparing two versions of a design
- Using the graphical interface: source, report, flow and project file editor
- Simulating the design with the Simulate tab and `wyrd sim`, and exporting a waveform
- Running the populated sample project
- Playing a level of Midgard III (Verilog Town), placing sensors and editing the Verilog of a traffic light

Every command, output and picture in this tutorial comes from a real run on one machine. The command output was pasted by a script (`docs/tutorial/shots/make_shots.py`); the pictures are grabs of the real windows, made by the scripts in the same folder, so the whole tutorial can be regenerated after a change. Two cosmetic changes were made: the project folder is shown as `~/wyrd-demo`, and long output is cut to the lines that matter where the text says so.

Conventions. Menu entries are written Menu1 ¿ Menu2 ¿ Item. Text you type in a terminal is shown after a `$` prompt. Numbers of runs vary from machine to machine only in the timing columns and in the run keys, which are hashes of the project path and tools.

1.1 What you need

Wyrd III runs on Linux, including WSL on Windows (WSLg shows the windows). You need Python 3.11 or later, a built Odin III 1.6.0 (the programs `odin3` and `odin3-flow`), and a VTR checkout for the `vpr` program and the architecture files. The timings in this tutorial are those of a laptop under WSL; every run in it takes about a second.

2 Installing and Configuring Wyrd III

2.1 Installing

Wyrd III is a Python package. In a clone of the repository, create a virtual environment and install it with the parts you want:

1. Create the environment and install the runner:

```
$ python3 -m venv .venv
```

```
$ .venv/bin/pip install -e .
```

2. Add the graphical interface (PySide6), and Bifrost III for simulation (the `odin3` Python package, version 1.0.3 or later; this tutorial was run with 1.6.0), and the game (pygame-ce):

```
$ .venv/bin/pip install -e ".[gui]"
$ .venv/bin/pip install -e ".[sim]"
$ .venv/bin/pip install -e ".[midgard]"
```

3. The `odin3` Python package is not on PyPI. Install the wheel of an Odin III release, or point `PYTHONPATH` at `odin3/plugins/python` and `ODIN3_LIB` at `libodin3.so` in an Odin III build.

4. Put the Odin III programs on `PATH`, and tell Wyrd III where VTR is:

```
$ export PATH="$HOME/odin3/build/release:$HOME/odin3/tools/flow:$PATH"
$ export VTR_ROOT=$HOME/vtr-verilog-to-routing
```

The commands this tutorial uses are `wyrd` (command line), `wyrd-gui`, `midgard` and `midgard-editor`.

2.2 Describing your machine

Where Odin III and VTR are, and how much memory Wyrd III may use, belong to the machine and not to a project. They live in one file, `~/.config/wyrd/machine.toml`. Wyrd III writes it for you:

1. Write the machine file with values detected on this machine:

```
$ wyrd config init
wrote ~/.config/wyrd/machine.toml
```

2. Look at the effective settings. Each line ends with where the value came from (here, the machine file):

```
$ wyrd config show
tools = {'odin3': 'odin3', 'odin3_flow': 'odin3-flow'}  (machine)
vtr_root = ~/vtr-verilog-to-routing  (machine)
vpr_gui = None  (default)
mem_budget = 15121374412  (machine)
jobs = 6  (machine)
launcher = systemd  (machine)
disk_min_free = 21474836480  (machine)
disk_check_path = /mnt/c  (machine)
vpr_mem = 4294967296  (machine)
other_mem = 2147483648  (machine)
history_factor = 1.25  (machine)
oom_retry_factor = 2.0  (machine)
step_timeout = 3600  (machine)
```

The memory budget (`mem_budget`) is 60% of the machine's memory by default: Wyrd III starts a step only when its memory estimate fits into what is left of the budget, and with the default launcher it caps each step with a `systemd-run` scope. The file can be edited by hand or from the graphical interface with Project ¿ Settings..., which shows one field for each key (Figure 1).

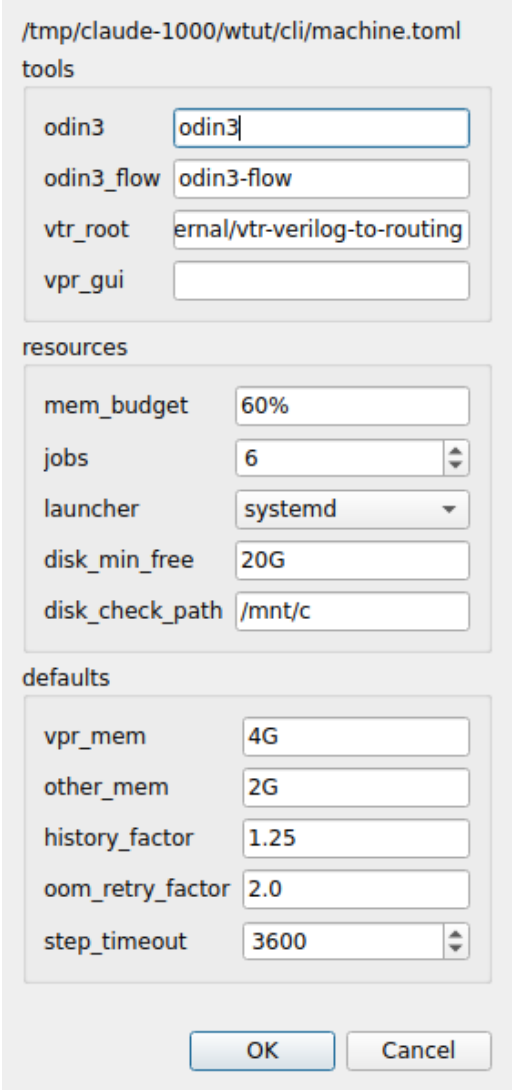


Figure 1. The Settings dialog edits the same machine file.

3 Making Your Own Project

A Wyrd III project is a folder with a file called `wyrd.toml`. In it you name your designs, say what to run on them (a recipe) and say which architectures to run on (a sweep). Everything Wyrd III produces goes into the same folder, so the project can be copied, archived or compared as a unit.

In this section we make a small design of our own, a counter plus an adder, and take it through the flow.

3.1 The Verilog design

The design adds a step you choose to the value of a free-running 4-bit counter: $total = count + step$. It has three modules in three files.

1. Make a folder for the project and a sub-folder for the sources:

```
$ mkdir -p ~/wyrd-demo/src
$ cd ~/wyrd-demo
```

2. Create `src/counter.v`, a counter with a synchronous reset:

```
// A 4-bit up counter with a synchronous reset.
module counter (input clk, input rst, output reg [3:0] count);
  always @(posedge clk) begin
    if (rst)
      count <= 4'd0;
    else
      count <= count + 4'd1;
  end
endmodule
```

3. Create `src/add4.v`, a combinational adder with a carry out:

```
// A combinational 4-bit adder with a carry out.
module add4 (input [3:0] a, input [3:0] b, output [4:0] sum);
  assign sum = a + b;
endmodule
```

4. Create `src/top.v`, which puts the two together:

```
// The counter value plus a step you choose: total = count + step.
module top (input clk, input rst, input [3:0] step, output [4:0] total);
  wire [3:0] count;
  counter u_counter (.clk(clk), .rst(rst), .count(count));
  add4    u_add     (.a(count), .b(step), .sum(total));
endmodule
```

Note. Do not name a module like a cell of the library. We first called the adder `adder`; Odin III refuses that name because `adder` is a hard cell it registered earlier (the carry-chain adder of the architecture). Section 3.3 shows the message.

3.2 Creating wyrd.toml

1. Ask Wyrd III for a commented project file in the current folder:

```
$ wyrd init ~/wyrd-demo
wrote ~/wyrd-demo/wyrd.toml
```

   The file it writes has a placeholder design (`CHANGE_ME.v`). Open `wyrd.toml` in your editor.

2. Change it to describe our project. The result is below.

```
[project]
name = "counteradd"

[[design]]
name = "counteradd"
files = ["src/*.v"]
top = "top"
```

```
[recipe.vtr]
steps = [{ use = "odin3-flow", target = "vtr", vpr = true }]

[sweep]
recipe = ["vtr"]
design = ["counteradd"]
arch = ["k6_frac_N10_frac_chain_mem32K_40nm", "EArch"]
```

The file has five parts:

- `[project]` gives the project a `name`.
- `[[design]]` describes one design: its `name`, its `files` (names or glob patterns, relative to this file) and its top module `top`. Without `top`, Odin III finds the top itself. You may have as many designs as you like.
- `[recipe.vtr]` is a recipe: an ordered list of steps. Our recipe has one step, `odin3-flow`, which runs Odin III, maps with ABC and runs VPR (`vpr = true`) on the architecture of the run.
- `[sweep]` says which combinations to run: every recipe, design and architecture in the three lists. Our sweep has $1 \times 1 \times 2 = 2$ runs. Any other key in this table is one more list to sweep. An architecture `X` is the file `$VTR_ROOT/vtr_flow/arch/timing/X.xml`.
- `[run]` holds experiment settings. Only `step_timeout` (seconds) is allowed; machine settings do not belong in a project.

3.3 When the Verilog is wrong: a hard-cell name

Odin III reports problems in a log with the file, line and column, and Wyrd III keeps the log with the run. Here is a real one. With the adder called `adder`, the flow stops in Odin III. Run the flow script by hand to see its message (`wyrd run` does exactly this for you and keeps the same log in the run directory):

```
$ odin3-flow src/adder.v src/counter.v src/top.v --top top --target vtr --no-vpr --out out
step odin3: FAILED (0.0 s) log ./out/odin3/odin3.log
  seconds          {'odin3': 0.004}
odin3_flow: step odin3 failed; see ./out/odin3/odin3.log
$ grep error out/odin3/odin3.log
odin3: error: ./src/adder.v:2:1: error: module 'adder' has the name of cell type 'adder' registered
odin3: error: pass read_verilog: failed: ODIN3_ERR_PARSE
odin3: error: -p: command 2: pass 'read_verilog' failed: ODIN3_ERR_PARSE
```

The cure is the one the message gives: rename the module. We call ours `add4`.

Note. A failed run is recorded with status `fail`. If a run fails very early, Wyrd III may remember a too small memory estimate for that design; if the next run then dies at once with no message, delete `wyrd.db` and `runs/` in the project, or set `launcher = "plain"` in `machine.toml`.

3.4 Running the project

1. Run the sweep (the `--progress` flag prints a line per finished run while it works):

```
$ wyrd run --progress
plan: 2 to run, 0 reused
counteradd/EArch [df2f30afbf69] ok
counteradd/k6_frac_N10_frac_chain_mem32K_40nm [c6e475febfbc] ok
ok counteradd EArch df2f30afbf69
ok counteradd k6_frac_N10_frac_chain_mem32K_40nm c6e475febfbc
```

Runs start smallest first. Each final line is the status, the design, the architecture and the first 12 characters of the run key.

2. Count the runs per status:

```
$ wyrd status
ok 2
```

The exit code of `wyrd run` is 0 when every run is `ok`, 1 when some run is not, and 2 for a usage, configuration or preflight error (a missing tool, a missing architecture file, too little disk space, another `wyrd run` in the same project).

3.5 Reading the results table

1. Print the table of the latest run of every design and architecture, keeping three metrics:

```
$ wyrd table --metrics luts,clbs,crit_path_ns
| design | arch | status | luts | clbs | crit_path_ns |
|---|---|---|---|---|---|
| counteradd | EArch | ok | 4 | 2 | 2.565 |
| counteradd | k6_frac_N10_frac_chain_mem32K_40nm | ok | 4 | 2 | 1.984 |
```

2. Ask for CSV when you want to paste into a spreadsheet or a script:

```
$ wyrd table --metrics luts,clbs,crit_path_ns --format csv
design,arch,status,luts,clbs,crit_path_ns
counteradd,EArch,ok,4,2,2.565
counteradd,k6_frac_N10_frac_chain_mem32K_40nm,ok,4,2,1.984
```

3. Without `--metrics` the table has one column for every metric the tool reported, in alphabetical order. That is the table the interface shows in Figure 6.

The metrics are the keys of the flow's `summary.json`: `luts` is the number of look-up tables, `clbs` the number of logic blocks, `hard_blocks.*` the hard blocks used (multipliers and memories), `min_chan_width` the minimum routing channel width, `crit_path_ns` the critical path in nanoseconds, `wirelength` the routed wire length and `seconds.*` the time spent in each tool. Our design has only four LUTs because its adder is mapped to the carry-chain cells of the architecture, which `luts` does not count. The two architectures give different critical paths and wire lengths for the same circuit; that is what a sweep is for.

3.6 What is in the run directory

Each run keeps everything it did. Every run has its own folder, and every step of the recipe a numbered folder inside it:

```
$ find runs -maxdepth 4
runs
runs/counteradd
runs/counteradd/EArch
runs/counteradd/EArch/df2f30afbf69
runs/counteradd/EArch/df2f30afbf69/0-odin3-flow
runs/counteradd/EArch/df2f30afbf69/replay.sh
runs/counteradd/k6_frac_N10_frac_chain_mem32K_40nm
runs/counteradd/k6_frac_N10_frac_chain_mem32K_40nm/c6e475febfbc
runs/counteradd/k6_frac_N10_frac_chain_mem32K_40nm/c6e475febfbc/0-odin3-flow
```

1. Look at the exact command Wyrd III ran. `cmd.sh` is a shell script you can run again by hand:

```
$ cat runs/counteradd/EArch/df2f30afbf69/0-odin3-flow/cmd.sh
#!/bin/sh
cd ~/wyrd-demo/runs/counteradd/EArch/df2f30afbf69/0-odin3-flow
export VTR_ROOT=~/vtr-verilog-to-routing
odin3-flow ~/wyrd-demo/src/add4.v ~/wyrd-demo/src/counter.v ~/wyrd-demo/src/top.v --top top --
```

2. Look at what the flow measured. These are the numbers in the table:

```
$ cat runs/counteradd/EArch/df2f30afbf69/0-odin3-flow/out/summary.json
{
  "luts": 4,
  "luts_note": ".names with two or more inputs (constants and one-input buffers or inverters e
  "clbs": 2,
  "hard_blocks": {
    "mult_36": 0,
    "memory": 0
  },
  "min_chan_width": 20,
  "crit_path_ns": 2.56528,
  "wirelength": 86,
  "seconds": {
    "odin3": 0.074,
    "vpr": 0.246
  }
}
```

3. The folder `out/` also holds `odin3/` (the netlist `top.blif` and `odin3.log`) and `vpr/` (`vpr.log` and the VPR run). `stdout.log`, `stderr.log` and `time.json` (wall time and peak memory of the step) are beside `cmd.sh`.

4. Write a script that replays all steps of a run, using a run key (or a prefix of at least six characters):

```
$ wyrd script df2f30afbf69
~/wyrd-demo/runs/counteradd/EArch/df2f30afbf69/replay.sh
```

```
$ cat runs/counteradd/EArch/df2f30afbf69/replay.sh
#!/bin/sh
# Replay re-runs the steps in place and overwrites their outputs in this run directory.
set -e
sh ~/wyrd-demo/runs/counteradd/EArch/df2f30afbf69/0-odin3-flow/cmd.sh
```

The results are also in `wyrd.db`, an SQLite file in the project, with a table of runs, of steps and of metrics.

3.7 Comparing two versions of a design

A project is a folder, so to compare two versions of a design we make a second project. We copy the folder to `~/wyrd-demo2` and make one change: the output of the adder is registered. The new `top.v` is

```
// The counter value plus a step you choose, registered: total = count + step one
    cycle later.
module top (input clk, input rst, input [3:0] step, output reg [4:0] total);
  wire [3:0] count;
  wire [4:0] sum;
  counter u_counter (.clk(clk), .rst(rst), .count(count));
  add4    u_add     (.a(count), .b(step), .sum(sum));
  always @(posedge clk) total <= sum;
endmodule
```

1. Run it, in the second folder:

   ```
   $ wyrd run
   ok counteradd EArch eed01cc087f0
   ok counteradd k6_frac_N10_frac_chain_mem32K_40nm 1da0339720f6
   ```

2. Compare the two projects. For every run that both have, the table shows the metric for A, for B and the ratio B/A:

   ```
   $ wyrd compare ~/wyrd-demo ~/wyrd-demo2 --metrics luts,clbs,crit_path_ns
   | design | arch | metric | A | B | B/A |
   |---|---|---|---|---|---|
   | counteradd | EArch | luts | 4 | 4 | 1 |
   | counteradd | EArch | clbs | 2 | 2 | 1 |
   | counteradd | EArch | crit_path_ns | 2.565 | 2.078 | 0.8101 |
   | counteradd | k6_frac_N10_frac_chain_mem32K_40nm | luts | 4 | 4 | 1 |
   | counteradd | k6_frac_N10_frac_chain_mem32K_40nm | clbs | 2 | 2 | 1 |
   | counteradd | k6_frac_N10_frac_chain_mem32K_40nm | crit_path_ns | 1.984 | 1.984 | 1 |
   ```

Registering the output cut the critical path of the `EArch` architecture to about 0.81 of its value. Runs that exist in only one of the projects are listed on the error output as `only in A:` or `only in B:`.

4 The Graphical Interface

`wyrd-gui` shows the same project in a window. It runs the flow as a `wyrd run` subprocess, so it does exactly what the command line does.

4.1 Opening the project

1. Start the window with the project folder as the argument:

   ```
   $ wyrd-gui ~/wyrd-demo
   ```

(Without an argument it opens the last project. You can also use File ¿ Open Project... or File ¿ New Project..., which runs `wyrd init` in a folder you choose.)

The window has the project tree on the left (designs with their files, recipes with their steps, and the architectures), four tabs in the middle (Source, Report, wyrd.toml, Simulate) and the docks Messages and Flow at the bottom. Figure 2 shows it with the source of `top.v`, which was opened by double-clicking the file in the tree.

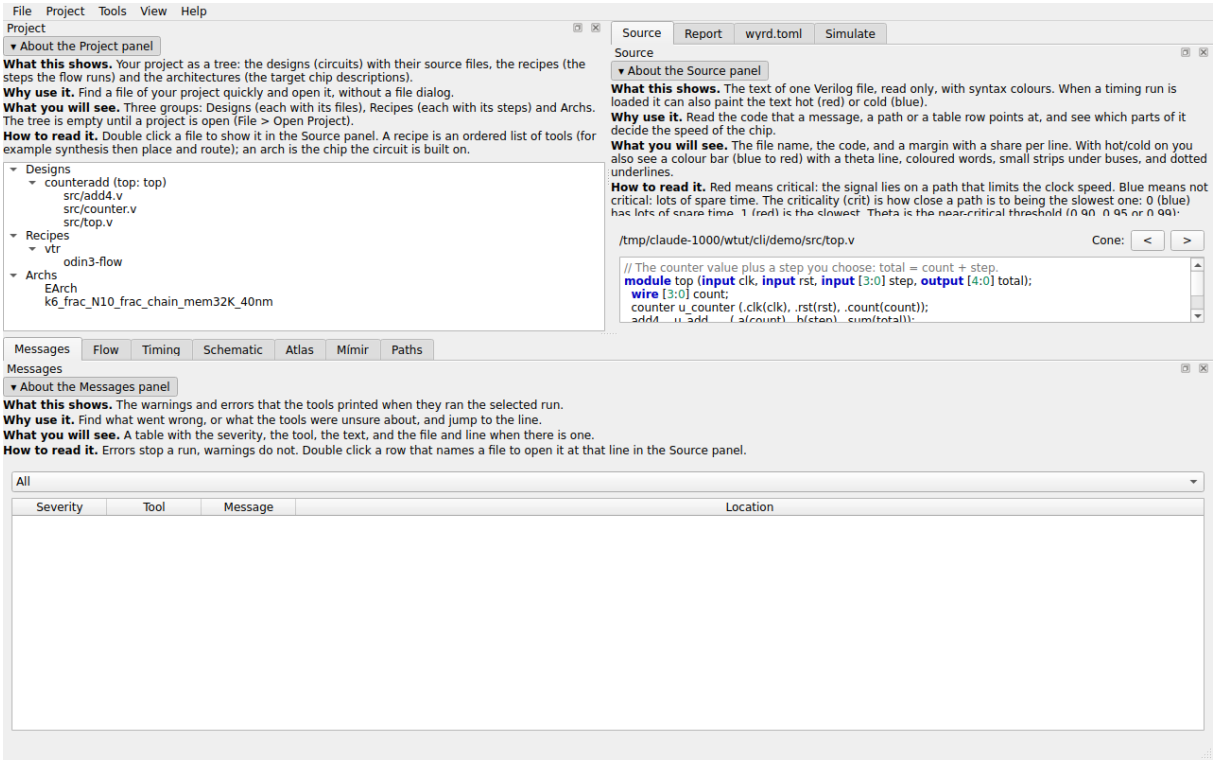


Figure 2. The project window: tree, source view and the Flow dock.

4.2 Editing wyrd.toml

1. Click the wyrd.toml tab (Figure 3). The editor checks the file while you type, with the same code that `wyrd run` uses. A valid file shows how many runs it expands to; an error shows its message and highlights the line. File ¿ Save wyrd.toml (Ctrl+S) saves it and reloads the project.

2. To add another design without typing TOML, choose Project ¿ Add design... (Figure 4). The dialog adds a `[[design]]` block, and optionally the name to the `[sweep]` list, to the editor text; nothing is written until you save.

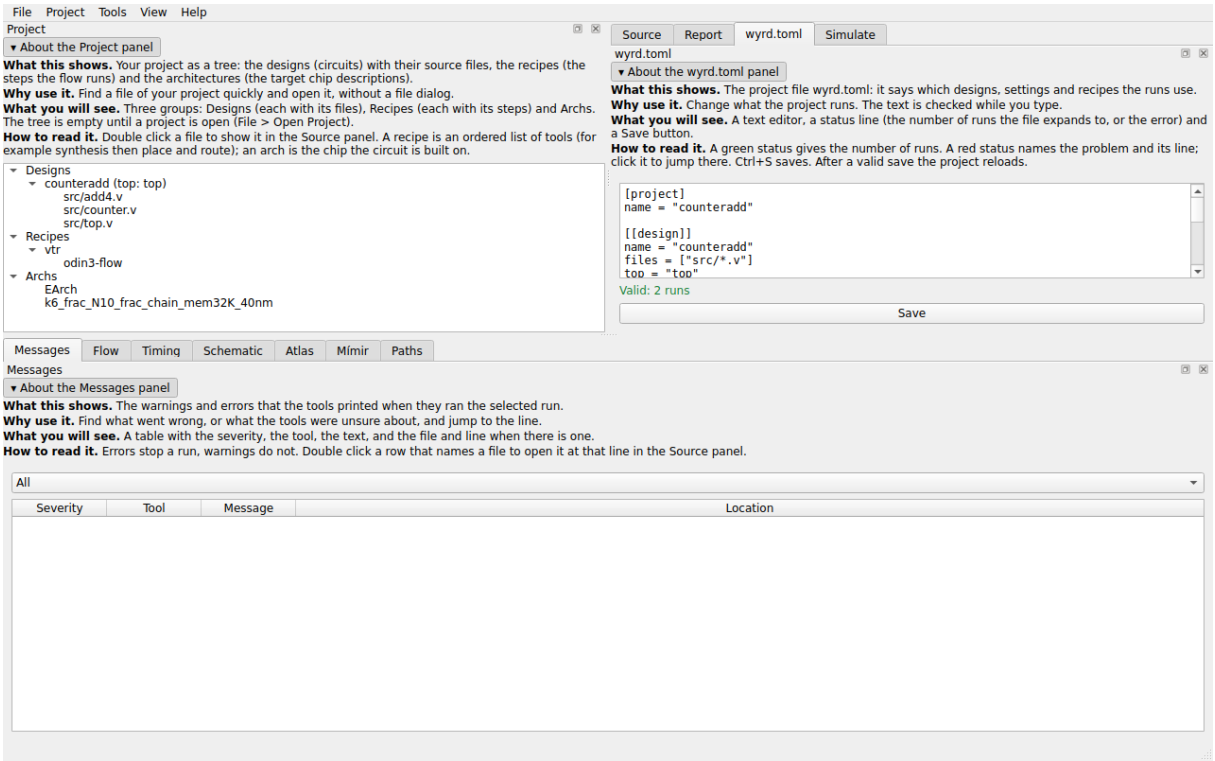


Figure 3. The wyrd.toml tab.

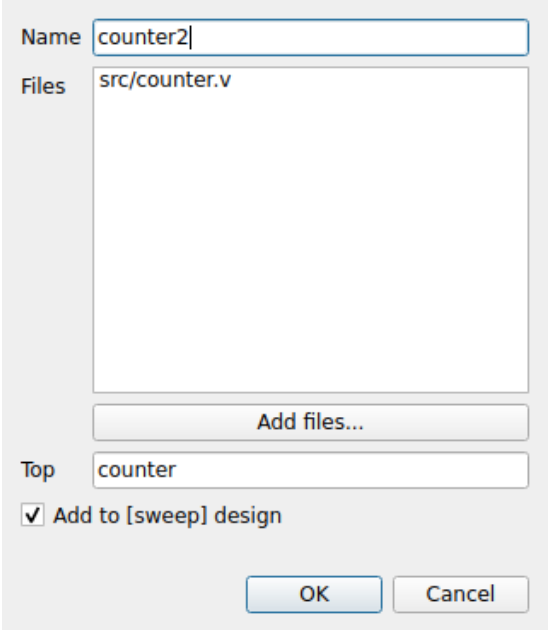


Figure 4. The Add design dialog (filled in here with a second design made of the counter alone).

4.3 Running the flow

1. Choose Project ¿ Run flow (Ctrl+R) or press Run in the Flow dock. Because we already ran the project on the command line, tick Force first, otherwise Wyrd III reuses the runs that are already `ok`. Figure 5 shows the flow while it works: the progress bar counts finished runs and the log shows the output of `wyrd run --progress`.

2. When it ends, the status line says all runs ok (Figure 6) and the Report tab shows the table of all runs.

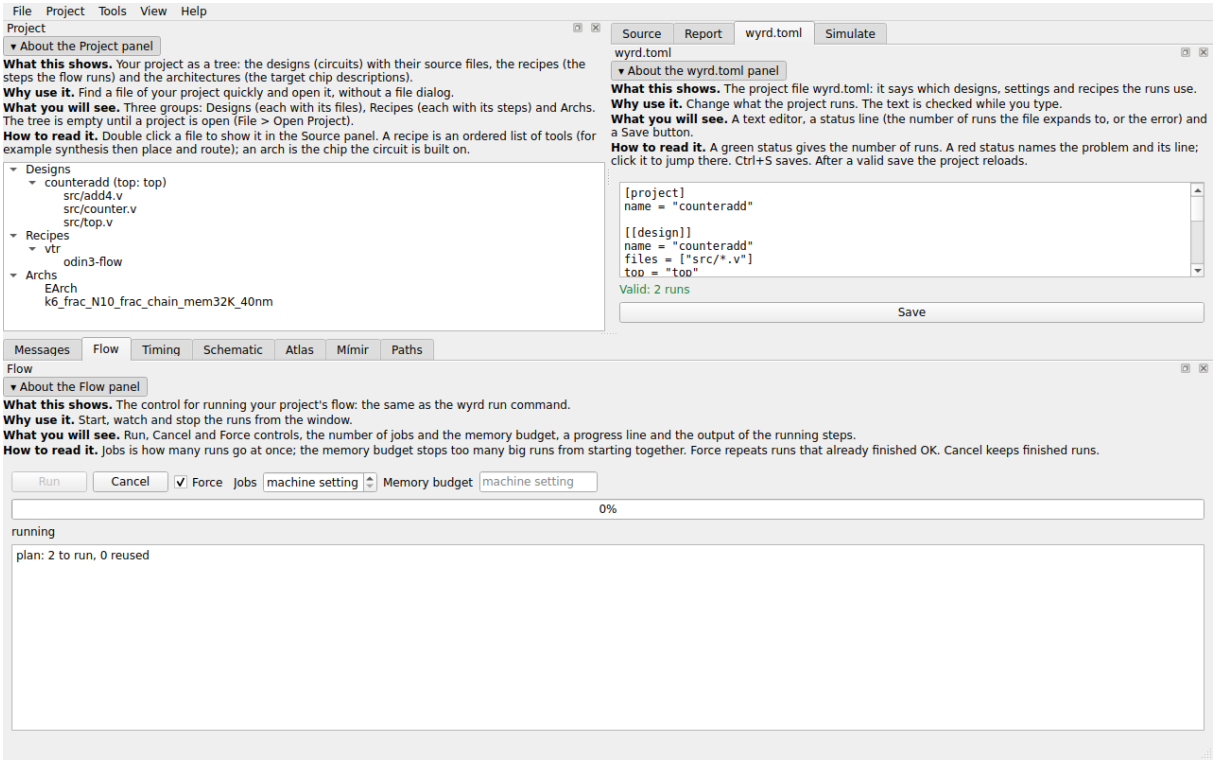


Figure 5. The Flow dock while the flow runs.

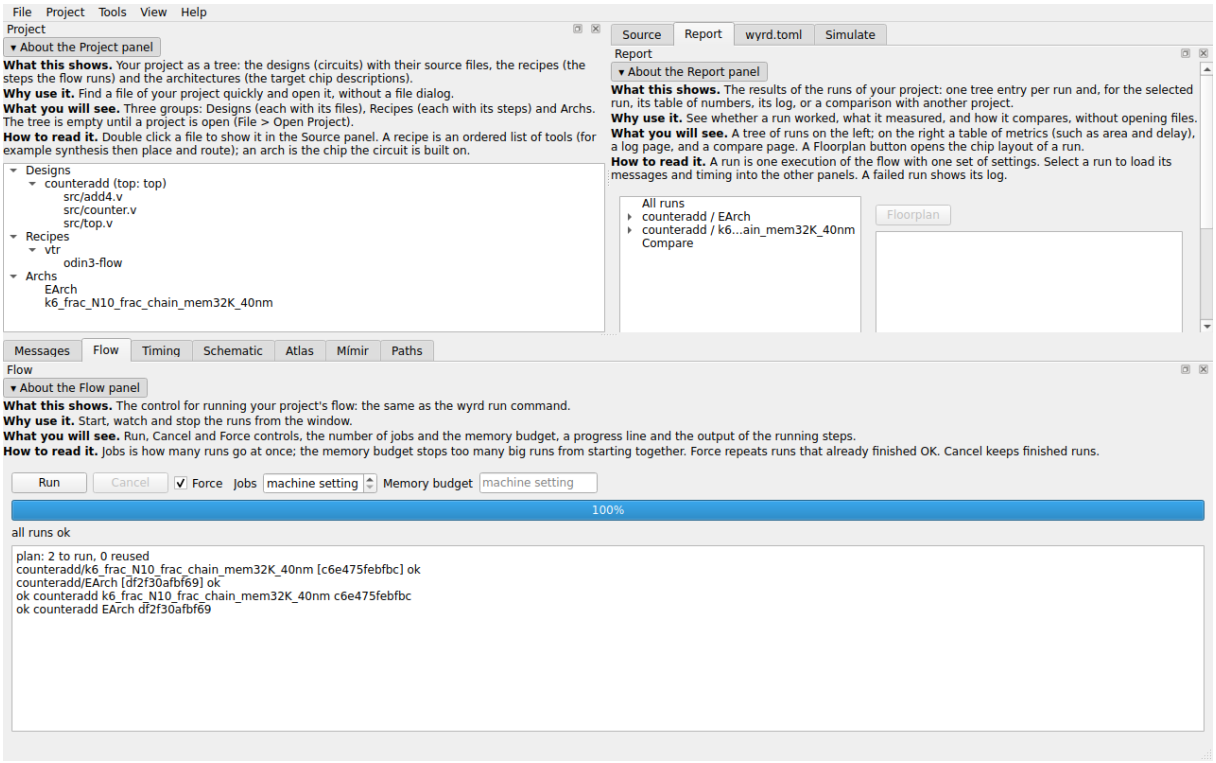


Figure 6. The flow has finished; the Report tab shows the table of all runs.

4.4 Reading the report

1. In the Report tab, open a run in the tree on the left and click Summary (Figure 7). The summary is a Quartus-style table: status, LUTs, CLBs, the hard blocks, Fmax (computed as $1000/crit_path_ns$), the minimum channel width, the critical path, the run directory and the run key.

2. Open Step 0: odin3-flow and click a log such as `odin3.log`. The Messages dock lists the warnings and errors Wyrd III found in the logs of the run, with the tool, the message and the place; double-click a row to open the file at that line (Figure 8).

3. The tree also has a Compare node, which does what `wyrd compare` does for another project folder, and a Floorplan button, which re-runs VPR with its graphics (it needs a VPR built with Qt6 graphics).

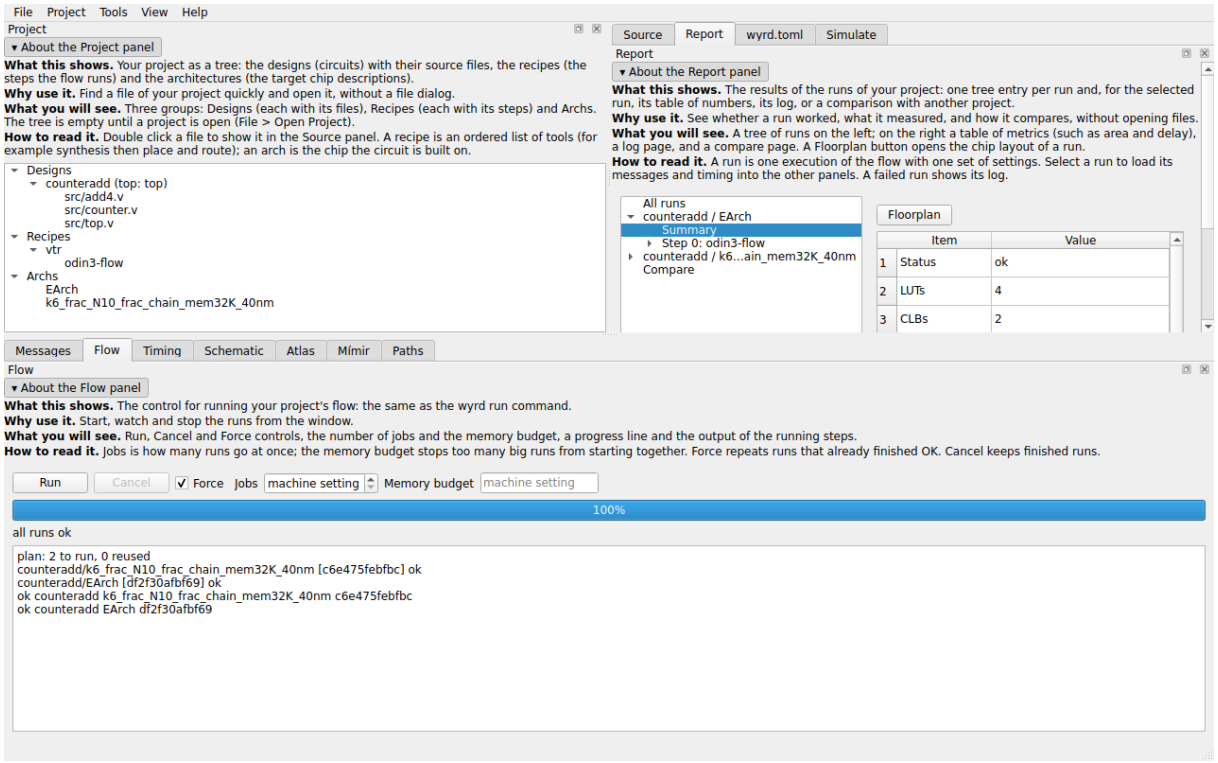


Figure 7. The Summary of one run.

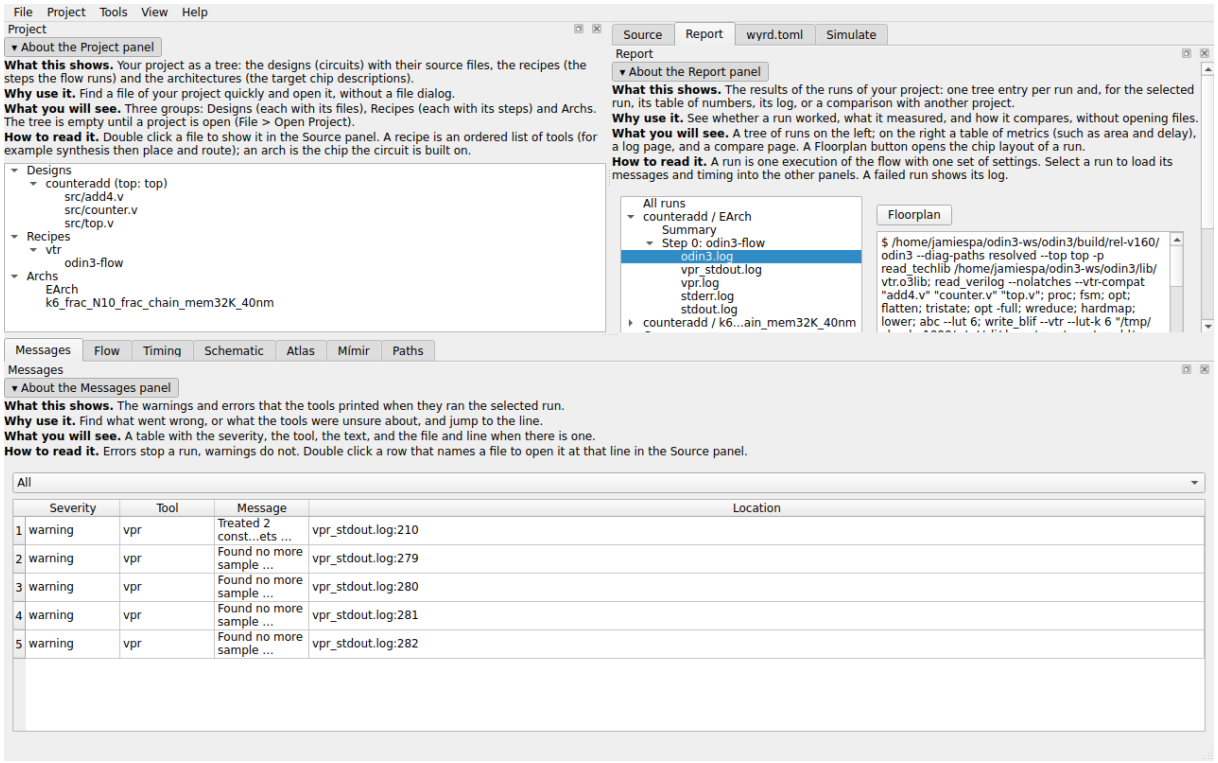


Figure 8. A log of the run, and the warnings Wyrd III found in it (Messages dock).

5 Simulating the Design

The flow in Section 3 only synthesises and implements the design. To see what it *does*, Wyrd III has Bifrost III, a wrapper around the per-cycle simulator of Odin III. One simulation step is one rising edge of the clock. It needs the `sim` extra of Section 2.

5.1 The floorplan viewer

The Floorplan button needs a VPR built with graphics (Qt6); the VPR of a normal VTR build is not. Build a second VPR out of tree and set it as `vpr_gui` under `[tools]` in `machine.toml`. The steps are in the user manual, section "Floorplan viewer (VPR graphics)" of the Getting started page. Batch runs keep the reference VPR; only the viewer uses the graphics one.

5.2 From the command line

1. Simulate with six random input vectors, write the waveform as a VCD file and save the vectors:

```
$ wyrd sim src/*.v --top top --random 6 --seed 3 --vcd counteradd.vcd --save-vectors counterad
1: rst=0 step=9 | total=10
2: rst=1 step=2 | total=2
3: rst=0 step=14 | total=15
4: rst=1 step=7 | total=7
5: rst=1 step=9 | total=9
6: rst=0 step=9 | total=10
```

   Each line is a clock cycle: the inputs applied, then the outputs after the clock edge. In cycle 1, `rst=0` and `step=9`: the counter has counted once, so `total` is $1+9=10$. In cycle 2 `rst=1` clears the counter, so `total` is $0+2=2$.

2. Look at the saved vector file. A line is `CYCLE INPUTS OUTPUTS` with every port written as binary digits, most significant bit first, in the order of the header lines:

```
$ cat counteradd.vec
# wyrd seed 3 cycles 6
# clock clk 0
# input rst 1
# input step 4
# output total 5
0 01001 01010
1 10010 00010
2 01110 01111
3 10111 00111
4 11001 01001
5 01001 01010
```

   You can edit such a file and run it again with `--vectors FILE`.

3. The beginning of the VCD file, which any waveform viewer reads (for example GTKWave):

```
$ head -24 counteradd.vcd
$version wyrd sim $end
$timescale 1ns $end
$scope module top $end
```

```
$var wire 1 ! clk $end
$var wire 1 " rst $end
$var wire 4 # step [3:0] $end
$var wire 5 $ total [4:0] $end
$upscope $end
$enddefinitions $end
#0
$dumpvars
0!
0"
b0000 #
b00000 $
$end
#2
b1001 #
#5
1!
b01010 $
#10
0!
#12
```

5.3 The Simulate tab

1. Click the Simulate tab or press Ctrl+T (Figure 9). Choose the design, optionally a top module, and press Compile.

2. After compiling, the table on the left lists every signal with its width and kind (Figure 10). Inputs can be edited: type a value in the Value column (decimal, or with the prefix `0x`, `0b` or `0o`). The clock is stepped, not set.

3. Set `rst` to 1 and press Step once to reset the counter. Then set `rst` to 0 and `step` to 9 and press Step N with 5 to run five cycles (Figure 11).

4. Change `step` to 3 and run ten more cycles. The Waveform tab draws the recorded signals (Figure 12); the Radix box chooses hex, decimal or binary. Run steps continuously at the chosen speed; Reset starts again.

5. To save the recording press Save VCD...; to open it in GTKWave press Open in GTKWave (Wyrd III uses `$WYRD_GTKWAVE` or `gtkwave` from `PATH`). Run random and Run vectors apply many cycles at once, as `wyrd sim` did.

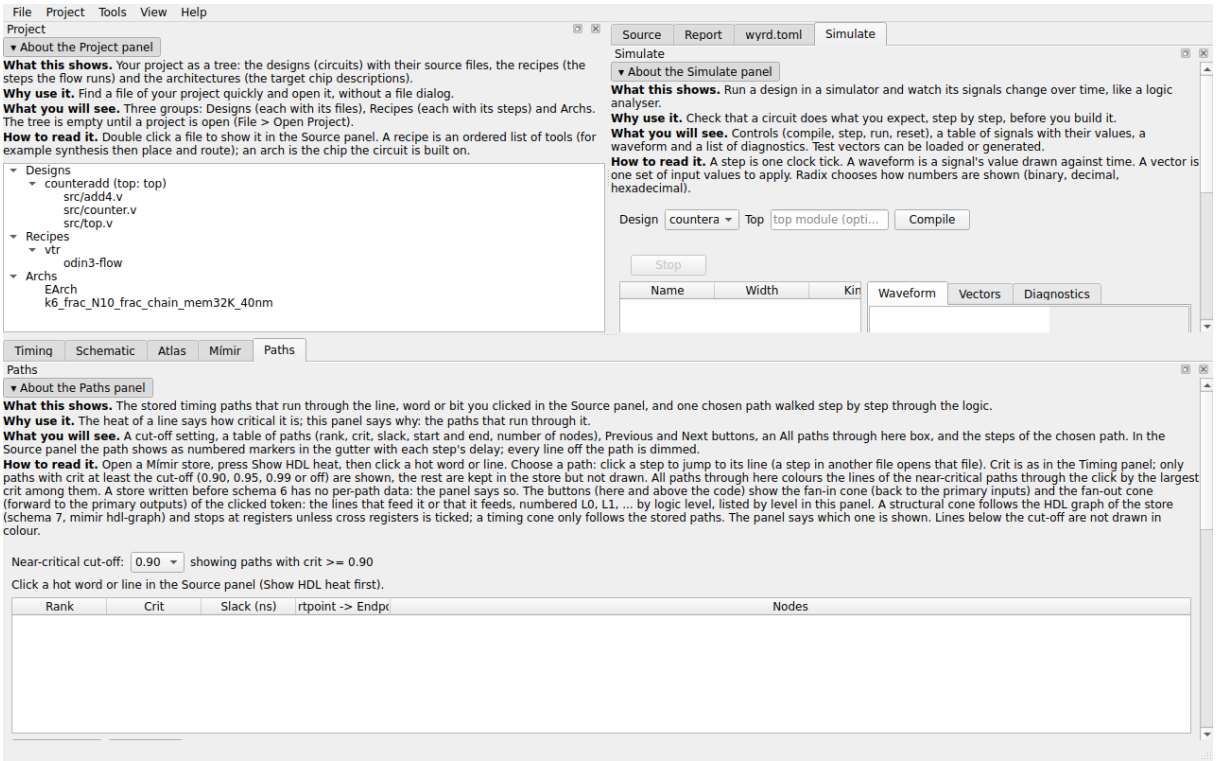


Figure 9. The Simulate tab before compiling.

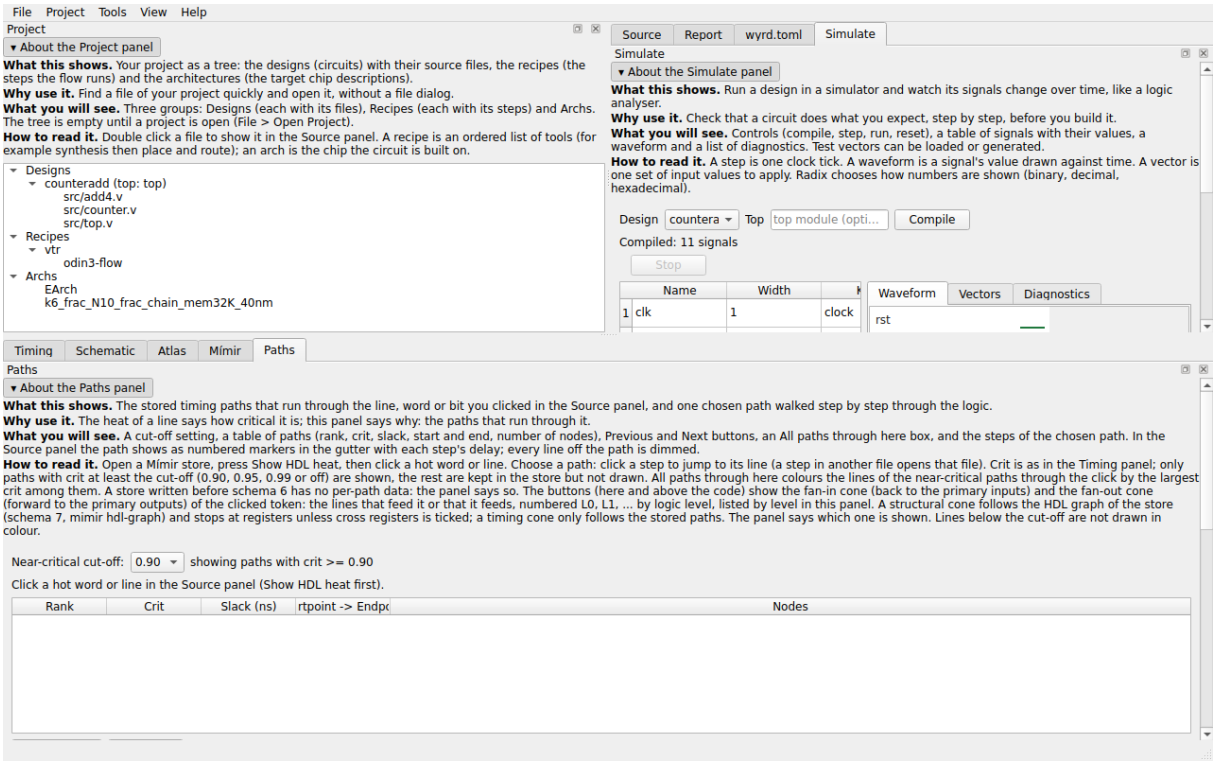


Figure 10. The design compiled: its four signals.

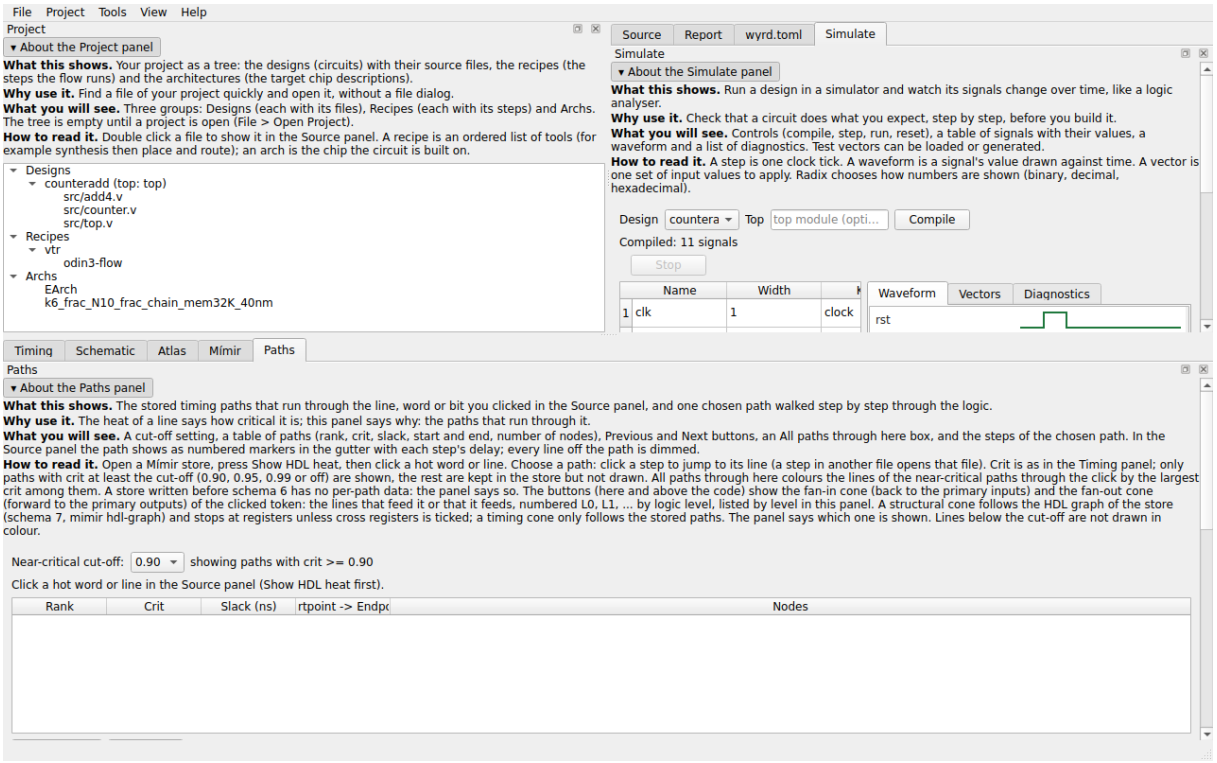


Figure 11. After a reset and five steps with `step` set to 9.

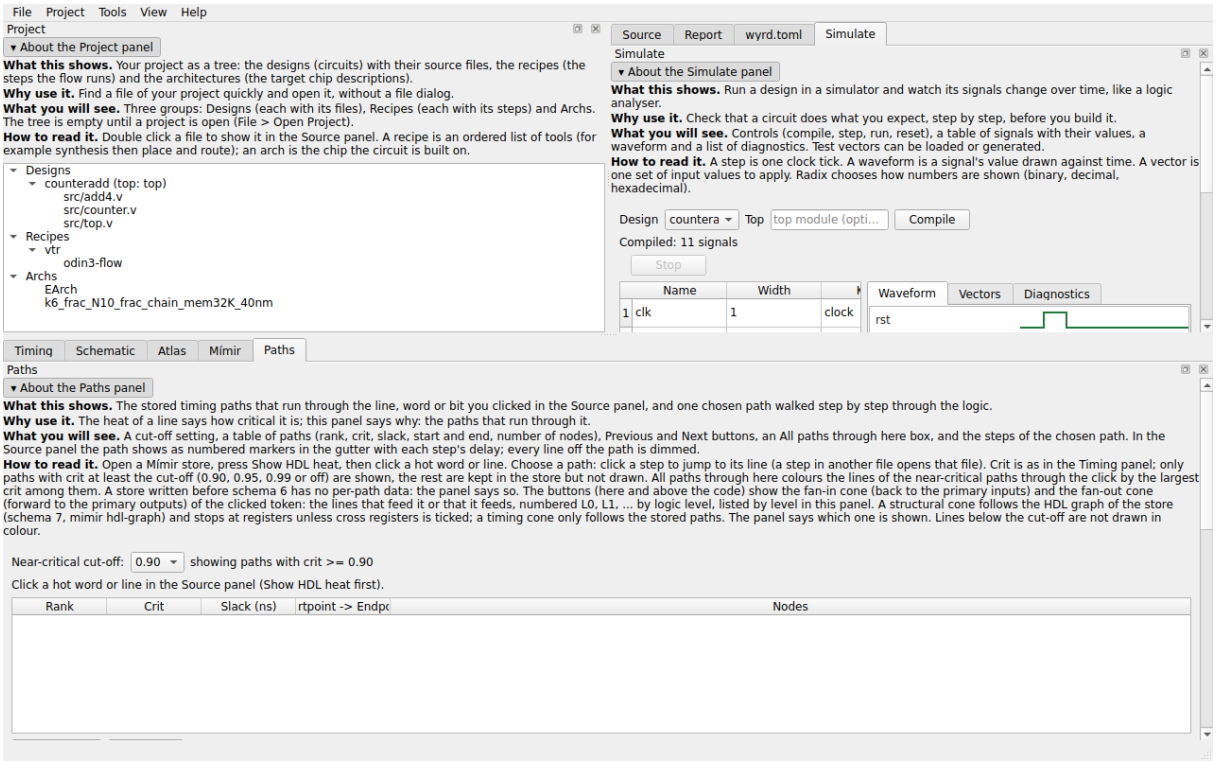


Figure 12. The waveform after ten more steps with `step` set to 3. `total` reads count + 3.

Note. The simulator limits are those of Odin III's simulator: a clock must be a top-level input; a signal that the optimiser removes must be marked `(* keep *)` in the Verilog to stay visible; a port that is both clock and data is not supported. The waveform samples once per step, so a combinational output changes at the rising edge in the picture.

6 The Populated Samples

Wyrd III ships a ready-made project, `samples/`, with three small designs: `single` (one file, a 2-to-4 encoder), `multi` (several files in folders) and `dsp` (a multiplier and a memory, which map to hard blocks). It sweeps them over the same two architectures: six runs.

1. Work on a copy, because the runs are written into the project folder:

```
$ cp -r samples ~/wyrd-samples
```

2. Run the sweep:

```
$ wyrd run --project samples
ok multi EArch f69172159268
ok single EArch d3141a1c665c
ok single k6_frac_N10_frac_chain_mem32K_40nm bc85e40ddd4c
ok multi k6_frac_N10_frac_chain_mem32K_40nm 67bfc63bd610
ok dsp EArch f3a87fa444e2
ok dsp k6_frac_N10_frac_chain_mem32K_40nm 2ac6e61383cc
```

   The smaller designs were started first.

3. Print the table. The `dsp` design uses one multiplier (`mult_36`) and one memory:

```
$ wyrd table --project samples --metrics luts,clbs,hard_blocks.memory,hard_blocks.mult_36
| design | arch | status | luts | clbs | hard_blocks.memory | hard_blocks.mult_36 |
|---|---|---|---|---|---|---|
| dsp | EArch | ok | 56 | 6 | 1 | 1 |
| dsp | k6_frac_N10_frac_chain_mem32K_40nm | ok | 56 | 6 | 1 | 1 |
| multi | EArch | ok | 5 | 1 | 0 | 0 |
| multi | k6_frac_N10_frac_chain_mem32K_40nm | ok | 5 | 1 | 0 | 0 |
| single | EArch | ok | 4 | 1 | 0 | 0 |
| single | k6_frac_N10_frac_chain_mem32K_40nm | ok | 4 | 1 | 0 | 0 |
```

4. Open the folder in the interface, `wyrd-gui  /wyrd-samples`, and look at the Report tab (Figure 13).

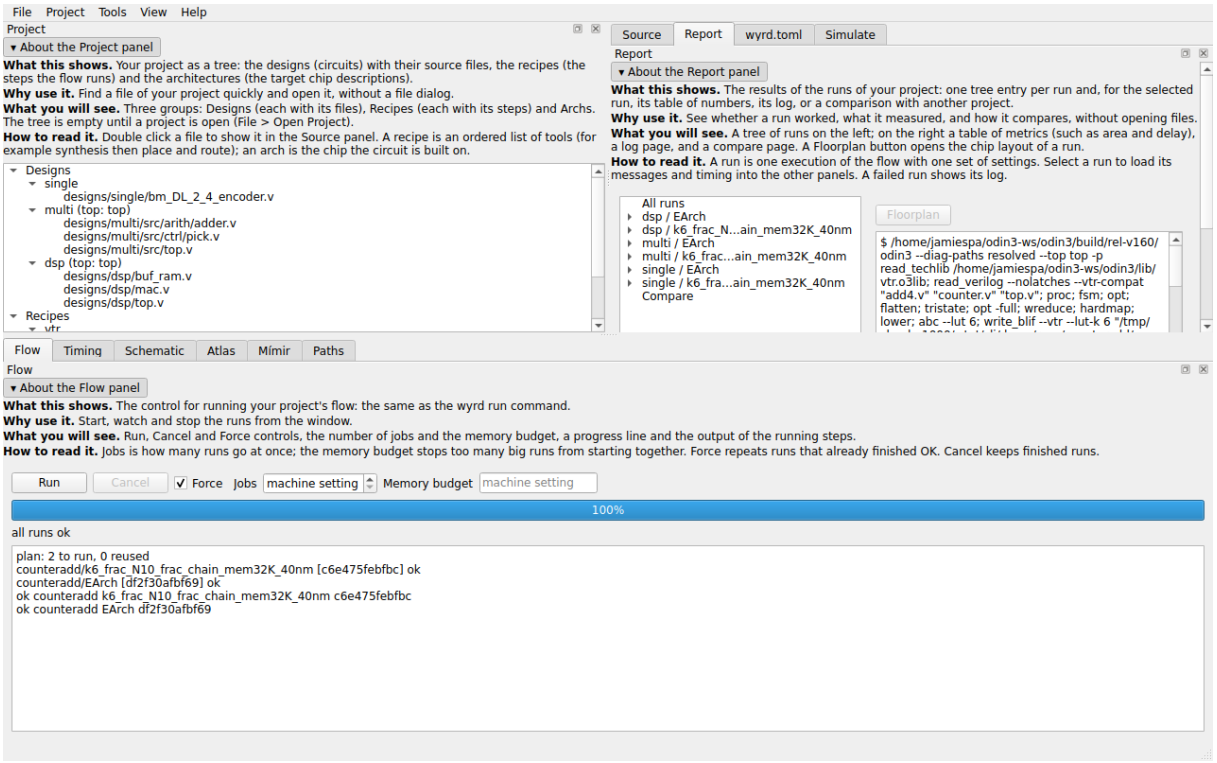


Figure 13. The sample project in the Report tab after the sweep.

The sample is also a model for your own project: copy a design folder, change `files` and `top` in `wyrd.toml`, and add or remove architectures in the sweep.

7 Midgard III (Verilog Town)

Midgard III is a game inside Wyrd III, a close replication of verilogTown (Jamieson, Grace, Mizuno, Zhang, ASEE 2017). You write one Verilog module for each traffic light of a small town, and cars drive through it. The modules are simulated by Odin III through Bifrost III. One game tick is one rising clock edge of your module; at speed 1x there are 60 ticks per second.

7.1 Starting the game

1. Start it from the command line, `midgard`, or from the interface with Tools ¿ Midgard III (Verilog Town). Figure 14 shows the menu: levels 1 to 12, the tutorial level (level 0), the high scores and the credits.

2. Choose level 1.

The files you own live in the data directory, `$MIDGARD_HOME` or `~/.local/share/midgard`. The game creates `Lv1/VerilogFiles/Traffic_signal_set_0.v` and one file for each other intersection from a starter module the first time you open the level.



Figure 14. The main menu (left) and level 1 in the edit phase (right). The sensor pile with a "6" is at the bottom-left of the map.

7.2 Placing a sensor

A level has seven general sensors, numbered 0 to 6. A sensor on a road tile tells your Verilog whether a car is on that tile and where it is going. Sensors can only be placed before the start (the edit phase).

1. Click the sensor pile at the bottom-left of the map. The highest-numbered unplaced sensor follows the mouse (Figure 15).
2. Click a road tile that is not part of an intersection. The sensor is placed and the sensor file is saved (Figure 15). Click a placed sensor to pick it up again; right-click removes it; the key C removes all.



Figure 15. Sensor 6 in the hand, and placed on a road tile north of the first intersection.

7.3 Editing the Verilog of a light

1. Click an intersection. The Verilog editor window opens with the file of that light (Figure 16). The module `light1` has the ports `clk`, `rst` (active low), `sensor_light` (8 bits: the entries and the inside of this intersection), `general_sensors` (30 bits: the sensors you placed), the four outputs `outN`, `outS`, `outE`, `outW` (3 bits: 0 Stop, 1 Forward only, 2 Left only, 3 Right only, 4 Go) and an optional `debug_port`.

2. For this tutorial we make one small change to the starter, so that the game shows us what the sensors deliver: the line `assign debug_port = sensor_light;` becomes `assign debug_port = general_sensors;`, and a comment line is added at the top. The pictures below were made after this change.

3. Press Verify (the green tick) to compile the file with Odin III (Figure 17).

4. Tick a box in the picture of the intersection, or type bits in the Internal Sensors field, and press Simulate (the green triangle). The editor steps the module one clock cycle and prints the outputs; here a car waits on the north entry (bit 6) and the light answers `OutN = 4`, Go (Figure 18).

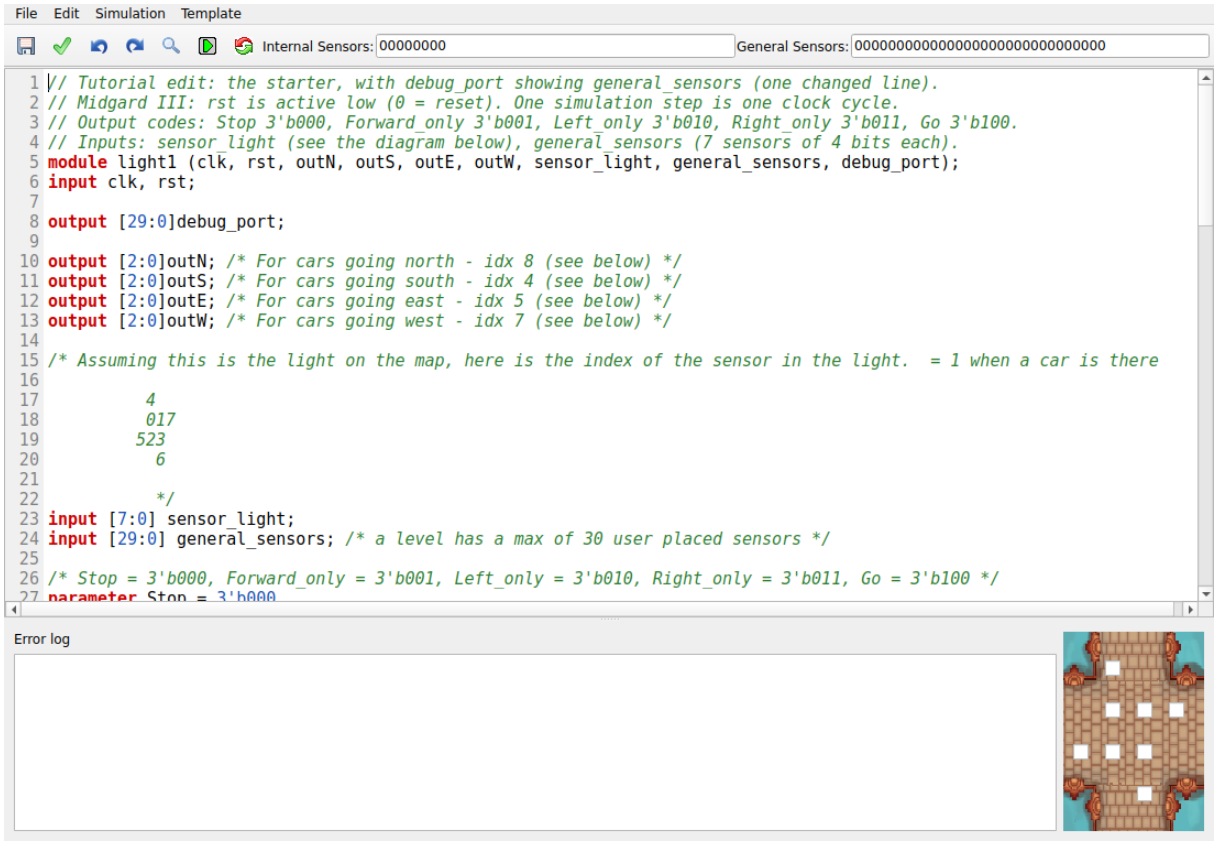


Figure 16. The Verilog editor with the file of the first light (the starter with the change of step 2).

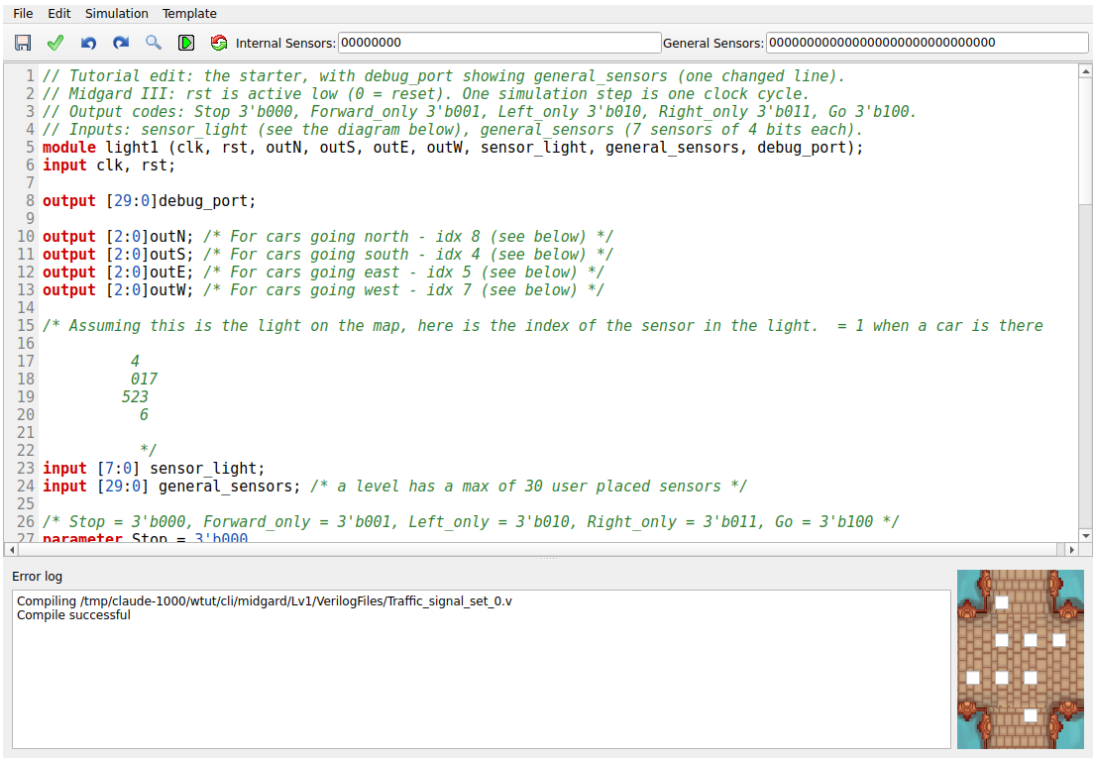


Figure 17. After Verify.

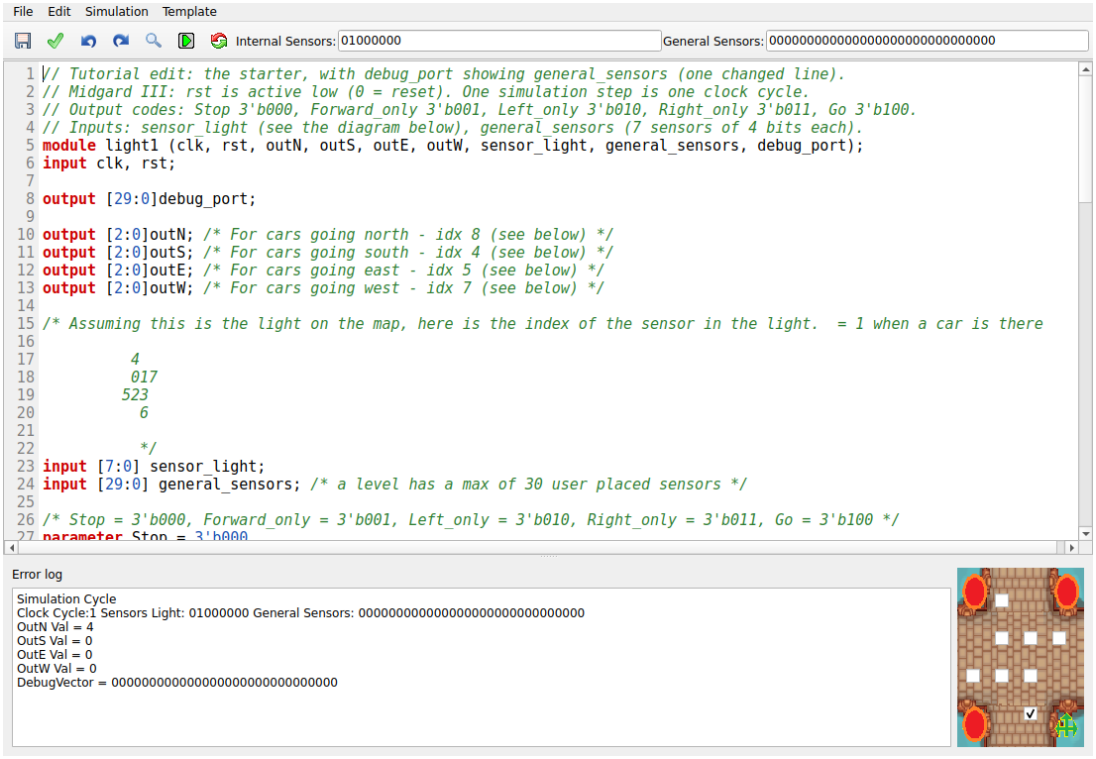


Figure 18. After Simulate with a car on the north entry.

7.4 Running the level

1. Close the editor and press S to start. Cars drive; the status line shows the speed and the tick rate; the buttons under the map change the speed, step and record.
2. Press P to pause when a car is on your sensor. In the pause the strip under the map lists, for every intersection, the inputs of the last tick and its four outputs (Figure 19). Here `general=0x0c000000`: sensor 6 (bits 27 to 24) reads 1100, a car is on its tile (bit 27) and the low bits of the ending counter of its destination are 100. Press S to resume.
3. When all cars are done the score screen shows the cars that passed and the time in ticks (Figure 20). The ten best results of every level are kept.

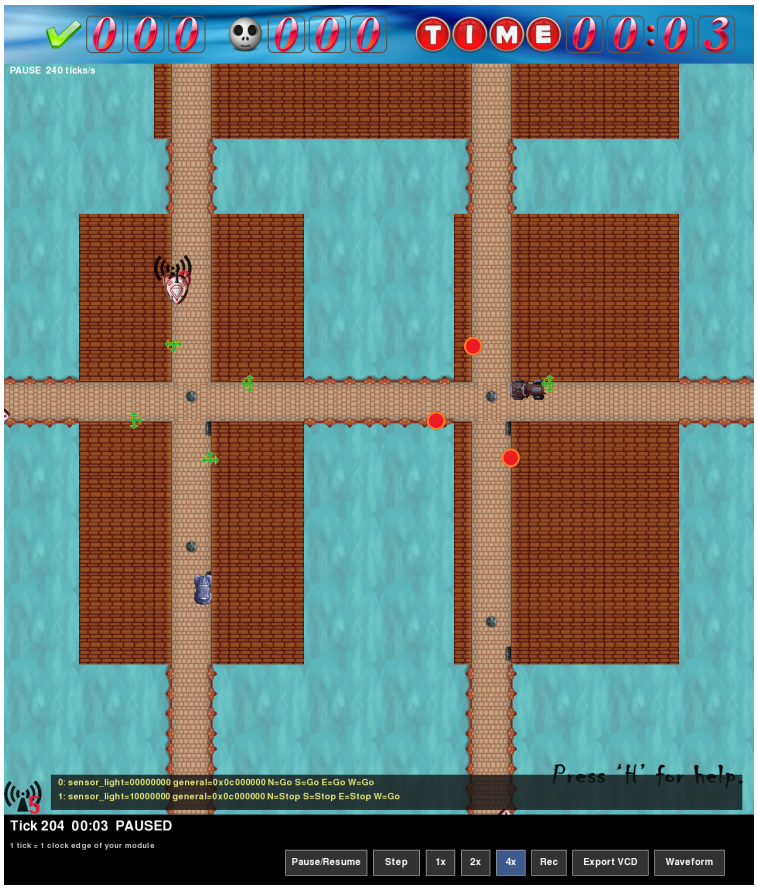


Figure 19. The level paused with a car on the sensor. The strip at the bottom shows the inputs and outputs of both intersections.



Figure 20. The score screen of level 1.

7.5 The level editor

You can make your own levels. Start `midgard-editor` (or Tools ¿ Midgard III Level Editor). The start dialog (Figure 21) lets you create a level of a chosen even size or update an existing level; Figure 21 shows level 1 loaded. You set the tile types, edit the cars with Edit Cars and Export the level as the XML file and the map picture the game reads.



Figure 21. The level editor: the start dialog (left) and level 1 loaded (right).

8 Where to Go Next

- The user manual (`docs/user`, built with Sphinx) has one page for each part: getting started, projects, running, the interface, simulation, Midgard III and the samples, with every command line flag and every key of `wyrd.toml`.
- The design documents are in `docs/specs`: the runner (W1), the interface (W2), Bifrost III (W3) and Midgard III (W4).
- The Odin III manual explains the synthesis flow that the `odin3-flow` step runs.